Design & Implementation of an NFC-Based Smart Drawer Lock

A Hybrid Authentication System (UID + APDU) with Mechanical Actuation

Ezz Nasr
Engineering Specialization: Computer Communications

February 9, 2026

Contents

Chapter 1

System Overview

1.1 Project Abstract

This project engineers a secure, invisible locking mechanism for a desk drawer using an ESP32 microcontroller. Unlike standard hobbyist locks, this system employs a **Hybrid Authentication Engine**. It provides instant access via standard NTAG213 tokens and encrypted, high-security access via EMV banking cards (Visa/-Mastercard) using APDU command parsing. Additionally, it integrates with the Apple ecosystem via a ”Controller-based” approach using iOS Shortcuts and Wi-Fi webhooks.

1.2 System Architecture Diagram

The system relies on a central controller (ESP32) managing peripherals via I2C (Reader) and GPIO (Actuator). The entire system is now powered by a single, stable 5V source.

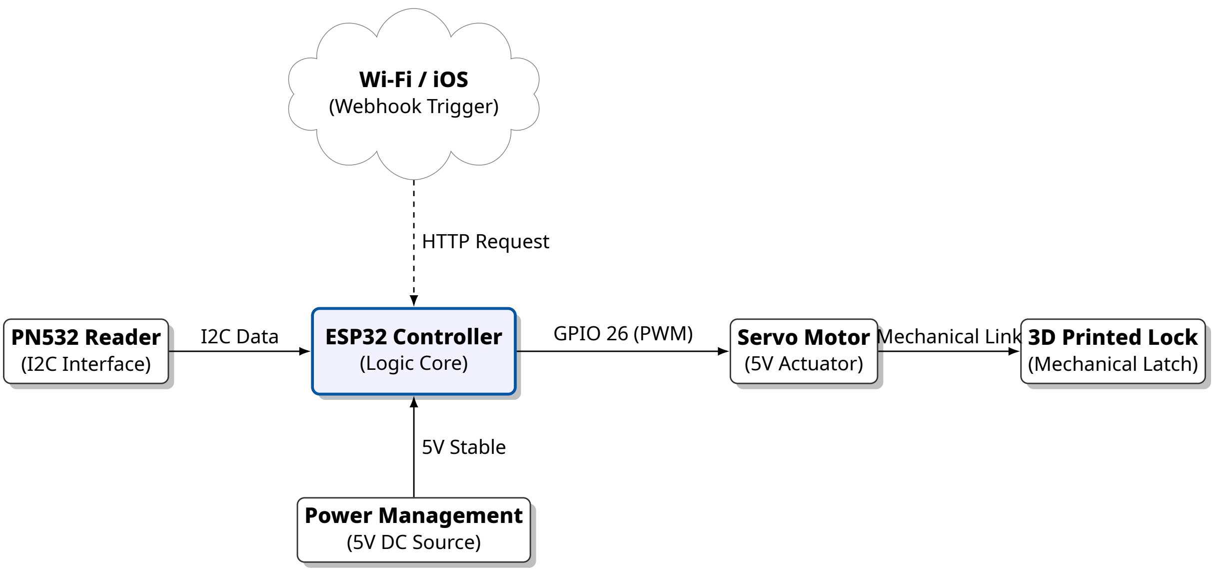


Figure 1.1: System Architecture Diagram

Chapter 2

Hardware Specifications

2.1 Bill of Materials (BOM)

Qty	Component	Description / Specification
1	**ESP32 Dev Module**	WROOM-32 (38-pin recommended)
1	**PN532 NFC Module**	Must support I2C mode
1	**Servo Motor**	SG90 (Plastic Gear) for light duty OR MG996R (Metal Gear) for heavy security
1	**5V Power Supply**	5V 2A USB Adapter will be enough tbh
1	**Capacitor (C_1)**	470μF or 1000μF, 10V+ Electrolytic-to avoid V-Drop Restarts-(Brownout)
1	**Diode (D_1)**	(EMF Protection)
1	**Piezo Buzzer**	5V Active Buzzer (for auditory feedback)
1	**Enclosure Materials**	PLA/PETG Filament for 3D printed lock body
-	**Wiring**	Jumper wires (Female-Female), Breadboard or PCB
4	**LEDs**	For visual sequences
1	**Break out Type C board**	for seamless power connection (only if PCB)
1	**rocker switch**	For Emergency power Off

Table 2.1: Complete Bill of Materials including electrical safety components

2.2 Microcontroller: ESP32-WROOM-32

The brain of the operation. Selected for its dual-core architecture (allowing network tasks to run without blocking NFC polling).

DATASHEET SUMMARY: ESP-WROOM-32	
Operating Voltage:	2.2V – 3.6V (Requires 3.3V Regulated)
Input Voltage (VIN):	5V (Directly from Power Source)
Wi-Fi Protocol:	802.11 b/g/n (Up to 150 Mbps)
Bluetooth:	v4.2 BR/EDR and BLE
GPIO Logic Level:	3.3V (NOT 5V Tolerant)
Max Current per GPIO:	12 mA (Source), 20 mA (Sink)

2.3 NFC Reader: PN532 (The "Red Board")

The most capable NFC chip for hobbyist use, supporting full Read/Write and Card Emulation.

DATASHEET SUMMARY: NXP PN532	
Frequency:	13.56 MHz (High Frequency)
Range:	5cm – 7cm (Antenna Dependent)
Operating Voltage:	2.7V – 5.5V
Current Consumption:	Typ. 100mA (Active), 30mA (Standby)

2.4 Actuator: Micro Servo Motor (SG90/MG996R)

The servo motor provides precise, low-power mechanical actuation for the 3D-printed lock. The choice between SG90 (micro) and MG996R (metal gear, high torque) depends on the mechanical resistance of the 3D-printed mechanism, in today's project i decided to use a light 3D printed lock as it made for domestic use and would be sufficient

DATASHEET SUMMARY: SG90/MG996R SERVO	
Operating Voltage:	4.8V – 6.0V (5V Nominal)
Control Signal:	Pulse Width Modulation (PWM)
Idle Current:	$\approx$ 10 mA
Stall Current (SG90):	$\approx$ 600 mA
Stall Current (MG996R):	$\approx$ 2.5 A
Actuation:	Rotational (typically 0° to 180°)

Chapter 3

Circuit Design, Wiring, and Electrical Hardening

3.1 Exact Wiring Table

The system is simplified to run entirely on a single 5V power rail. The servo motor's power lines **MUST** be connected directly to the 5V power source, not through the ESP32's on-board regulator, to prevent brownouts. an Alternate approach would be using a commercially available solenoid lock, although it would require a step up converter

Component	Pin Mapping
Power Input	5V DC Adapter → Common 5V Rail 5V DC Adapter → Common GND Rail
ESP32 Power	Common 5V Rail → ESP32 VIN Common GND Rail → ESP32 GND
NFC Reader (I2C)	ESP32 3V3 → PN532 VCC ESP32 GND → PN532 GND ESP32 GPIO 21 → PN532 SDA ESP32 GPIO 22 → PN532 SCL
Servo Motor	Common 5V Rail → Servo VCC (Red Wire) Common GND Rail → Servo GND (Brown/Black Wire) ESP32 GPIO 26 → Servo Signal (Orange/Yellow Wire)
Feedback (Buzzer)	ESP32 GPIO 18 → Buzzer (+) ESP32 GND → Buzzer (-)

Table 3.1: System Pinout Configuration on the bread Board

3.2 Detailed Wiring Diagram with Electrical Hardening

The servo motor is an inductive load and can introduce noise and voltage spikes into the power rail, potentially causing the ESP32 to reset (brownout). The following schematic uses standard electrical symbols to illustrate the critical components required to protect the microcontroller and ensure stable operation.

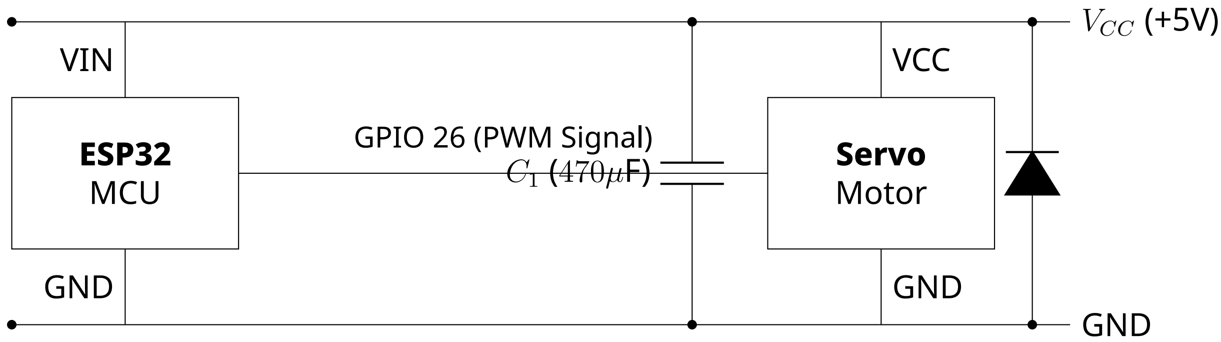


Figure 3.1: Schematic of Circuit with precautions against EMF

3.3 Electrical Safety and Component Hardening

The transition to a servo motor, while simplifying the power supply, introduces new electrical noise challenges due to the motor's rapid current draw and back-EMF.

- **Bulk Capacitor (C_1):** A large electrolytic capacitor (e.g., **470 μF, 10V**) must be placed **as close as possible** to the servo motor's power pins (VCC and GND).

  i Logic Insight

  Purpose: Servos draw large, short bursts of current (up to 2.5A for an MG996R) when they start moving or encounter resistance. This causes a momentary voltage drop (brownout) on the 5V rail, which can reset the ESP32. The capacitor acts as a local, high-speed energy reservoir to supply these current spikes, isolating the ESP32 from the noise.

- **Diode** should be placed across the 5V power rail, with the cathode (banded side) connected to V_{CC} and the anode to GND, acting as an OC when the servo is ideal, and being a SC to dissipate the excess current generated due to the EMF

i Logic Insight

Purpose: Although a servo is controlled by a closed-loop circuit, its internal DC motor is an inductive load. When the motor is suddenly stopped or reversed, it generates a voltage spike (back-EMF) that can exceed the supply voltage and damage the ESP32 or the power supply. The reverse-biased diode provides a safe path for this transient current to recirculate, clamping the voltage spike and dissipating the energy harmlessly.

- **Power Rail Separation:** The servo's VCC and GND lines **MUST** be connected directly to the main 5V power source, not to the 5V pin on the ESP32 board. This ensures the high current draw bypasses the thin traces and potentially weak regulators on the microcontroller board.

3.4 Emergency Fail-Safe Mechanism

a drawback of the current implementation is that in the event of power loss, the door or drawer locked wouldn't be accessible at all.
proposed solutions :

- 1- make a mechanical way to bypass the ESP32 entirely (like physically moving the latch with a string)
- 2- implementing a battery solution for emergencies.

  we will not be diving into these solutions at this time.

Chapter 4

Hybrid Authentication Engine

This section details the "If/Else" logic that allows the system to differentiate between a simple tag and a complex credit card.

4.1 The Logic Flow

The code must implement a non-blocking state machine.

- **Path A (Fast):** Reads the UID. If it matches the whitelist, UNLOCK immediately (Time: $<$ 200ms).
- **Path B (Slow):** If UID is random/unrecognized, assume it is a Banking Card. Initiate APDU handshake to extract the PAN (Primary Account Number). (Time: 1.5 – 2.0s).

i Logic Insight

ID is short for Uniqe ID , that is present in NFC chips such as the normal cards that come with the NFC reader or **Credit Cards**

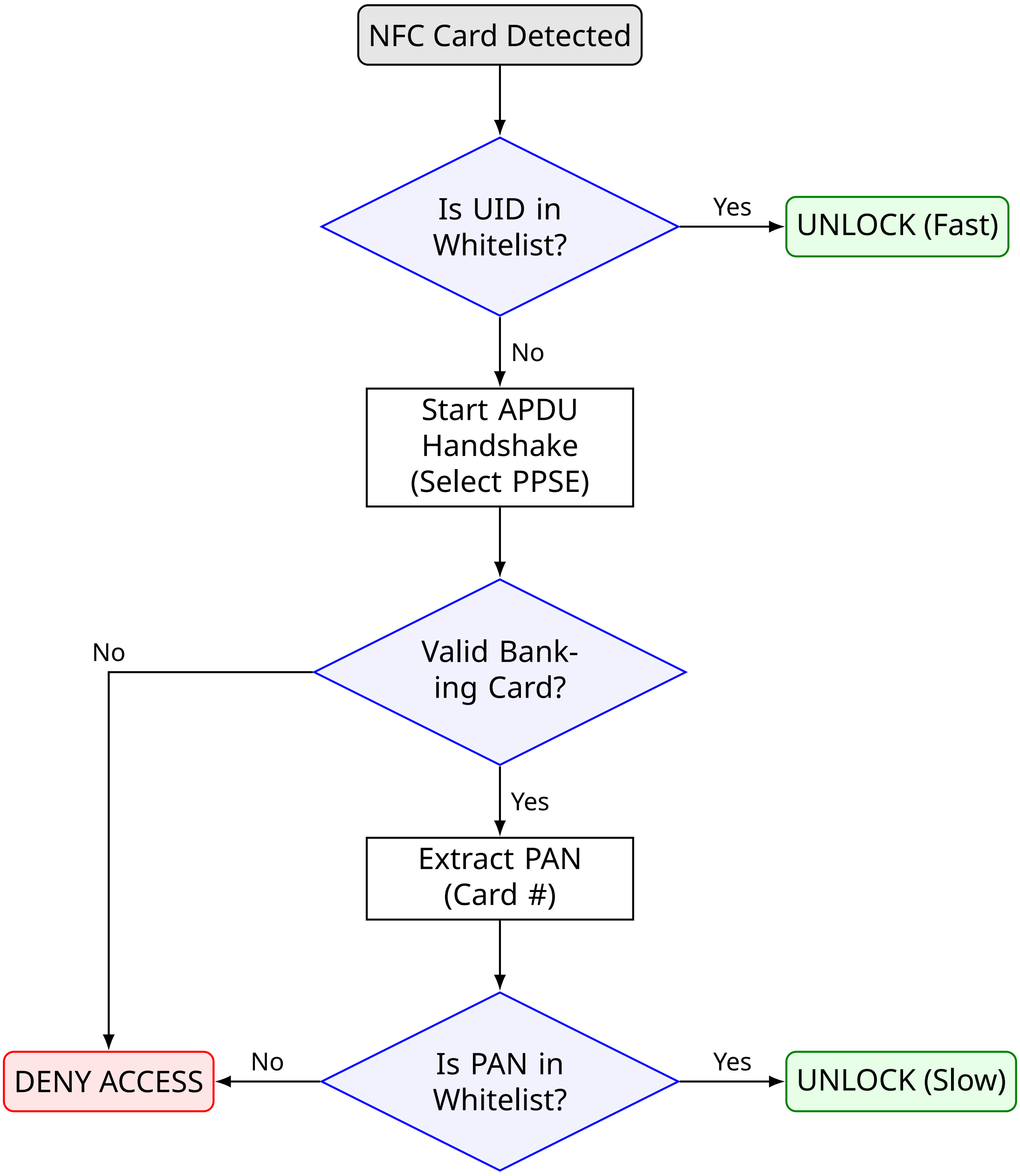


Figure 4.1: Hybrid Authentication Logic Flowchart

4.2 The Credit Card Handshake (Technical Detail)

Why does it take 2 seconds? Because we are not just reading a serial number; we are negotiating a session.

1. **SELECT PPSE (2PAY.SYS.DDF01):** The ESP32 asks the card, "What payment methods do you support?"
2. **SELECT AID:** The card replies with a list (e.g., Visa ID: A0000000031010). The ESP32 must select the specific App ID.
3. **GET PROCESSING OPTIONS (GPO):** The ESP32 asks for the file processing structure.

4. **READ RECORD:** The ESP32 reads the specific memory block containing the card number (PAN).

Chapter 5

Apple Ecosystem Integration

5.1 The "Controller" vs. "Key" Concept

//

i Logic Insight

Constraint: Apple devices restrict direct NFC UID access (CoreNFC limitations). We cannot use the iPhone as a passive "Key."
Solution: We invert the relationship. The iPhone acts as the "Controller." It reads a passive tag/sticker on the desk and sends a command to the lock via Siri Shortcuts.

similar logic can be applied for an android for to achieve the same results

5.2 Implementation Steps

1. **Hardware:** Stick a standard NTAG213 sticker on top of the drawer (hidden or visible).
2. **ESP32 Setup:** Program the ESP32 to listen for a specific HTTP Webhook (e.g., `http://192.168.1.50/unlock`).
3. **iOS Shortcut:** Create a "Personal Automation" in the Shortcuts app:
   - **Trigger:** "When I tap NFC Tag..."
   - **Action:** "Get Contents of URL" $\rightarrow$ (Your ESP32 IP).
   - **Option:** "Run without asking" (Enabled).

Chapter 6

Phased Implementation Plan

Phase 1: The ”Open Air” Prototype and Mechanical Design

- Assemble components on a breadboard, including the servo and electrical hardening components (C_1, D_1).
- **Goal:** Read a standard tag UID via Serial Monitor and successfully actuate the servo motor between its locked and unlocked positions.
- **Verification:** Ensure the servo moves smoothly and the ESP32 does not reset during actuation.
- **Mechanical Design:** Finalize the 3D-printed lock mechanism, ensuring minimal friction and incorporating the mechanical override.

Phase 2: Power & Electrical Validation

- Connect the 5V supply and test the system under load.
- **Load Test:** Run the servo continuously for 5 minutes. Monitor the 5V rail voltage with an oscilloscope or multimeter to ensure stability (no drops below 4.5V).
- **Code Safety:** Implement the ”Watchdog Timer” (Force servo to stop after 5 seconds of continuous running) to prevent motor burnout or mechanical binding.

Phase 3: The Hybrid Code Integration

- Introduce the APDU library.
- **Goal:** Successfully read a Credit Card PAN and output it to Serial.
- **UX Tuning:** Tune the buzzer to beep *during* the read process so the user doesn’t pull the card away too early.

Phase 4: Enclosure & Final Mounting

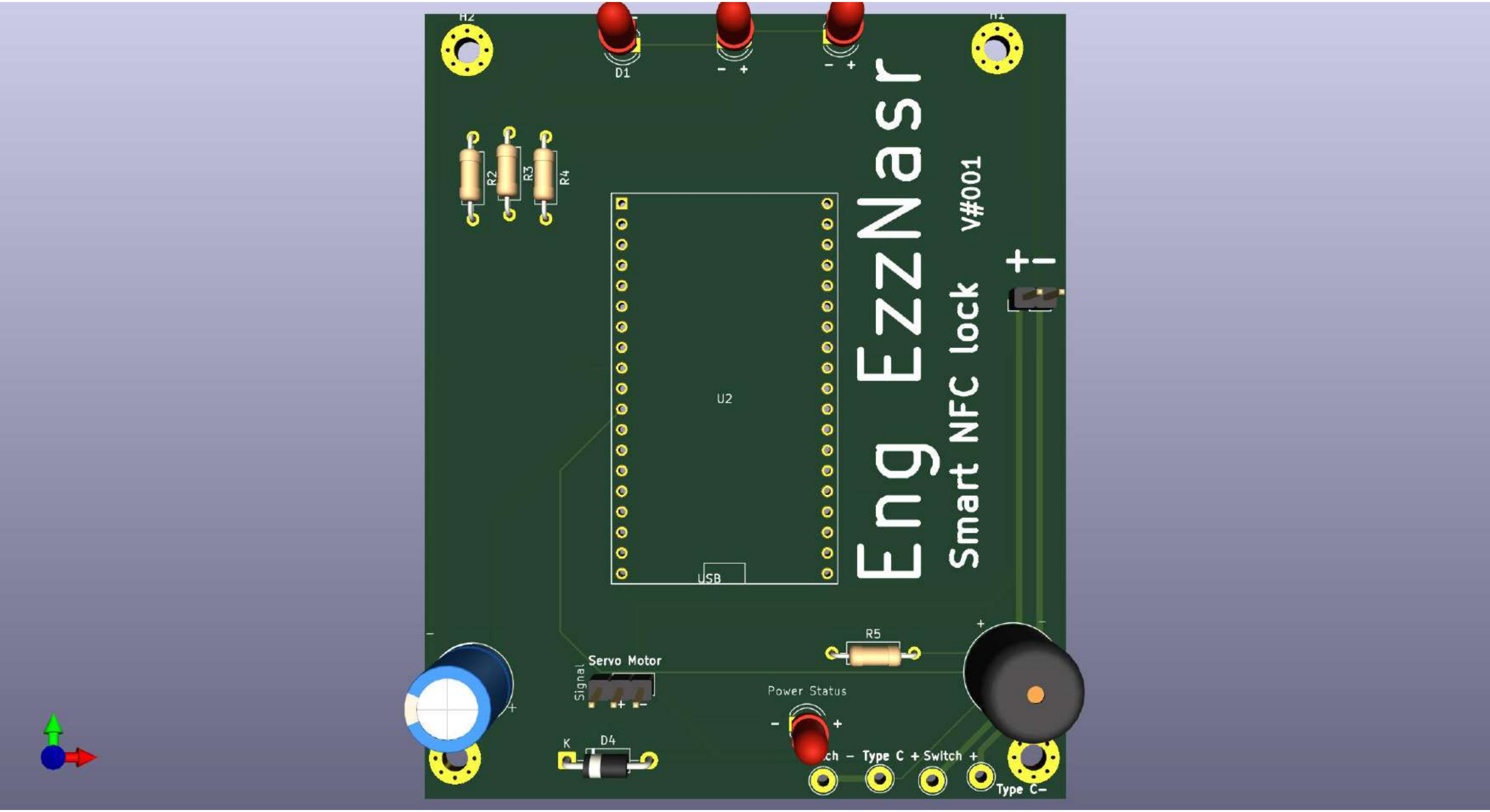
- 3D print the final housing and mechanical parts.
- **Critical Step:** Install the Mechanical Override (Fail-Safe).
- Mount the reader using nylon screws (no metal!) to preserve antenna tuning.

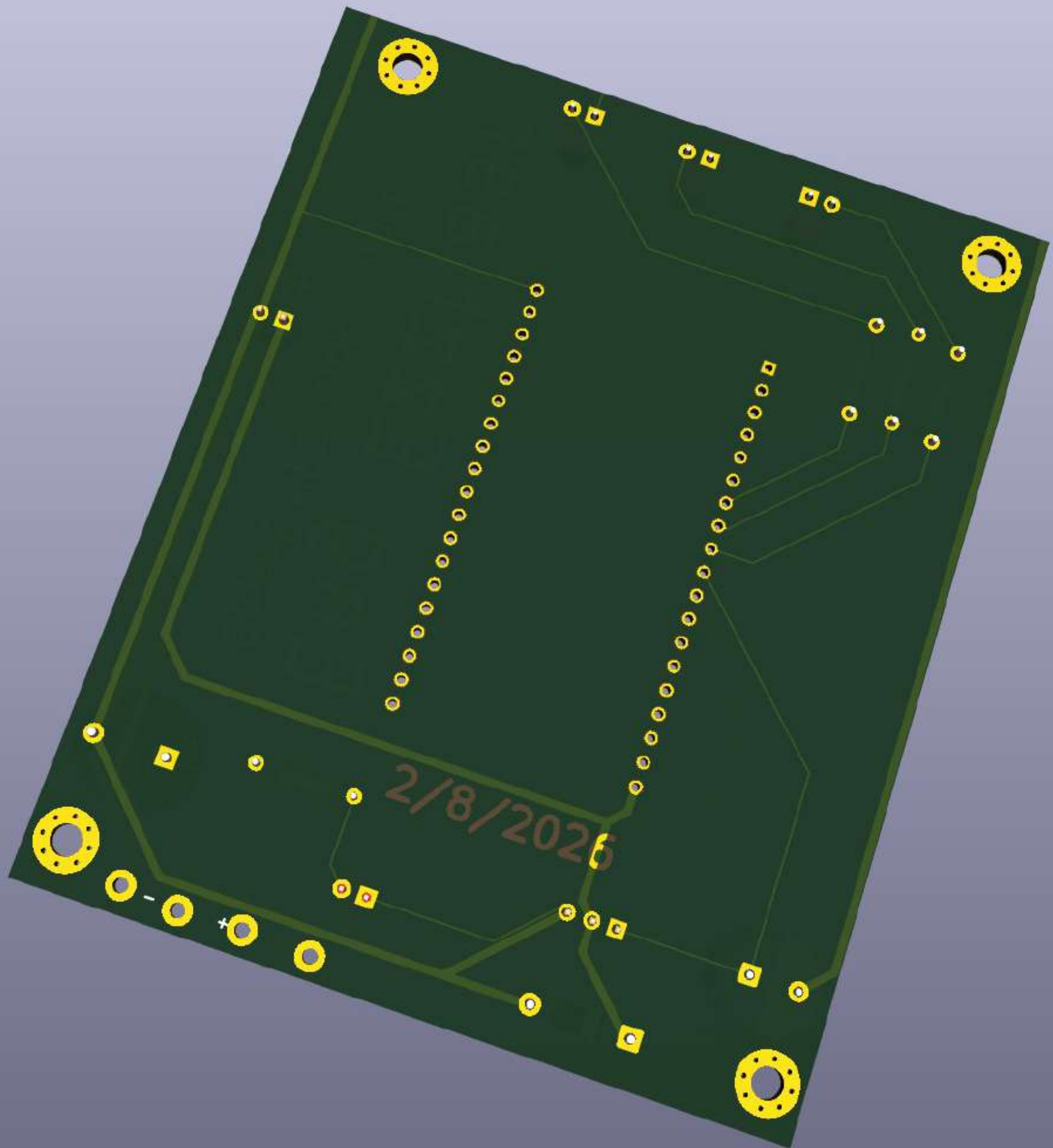
//

i Logic Insight

Note that along with this file there will be a Code file along with the 3D files ready to print for the convenience of the reader
for those of you who will be building the PCB board

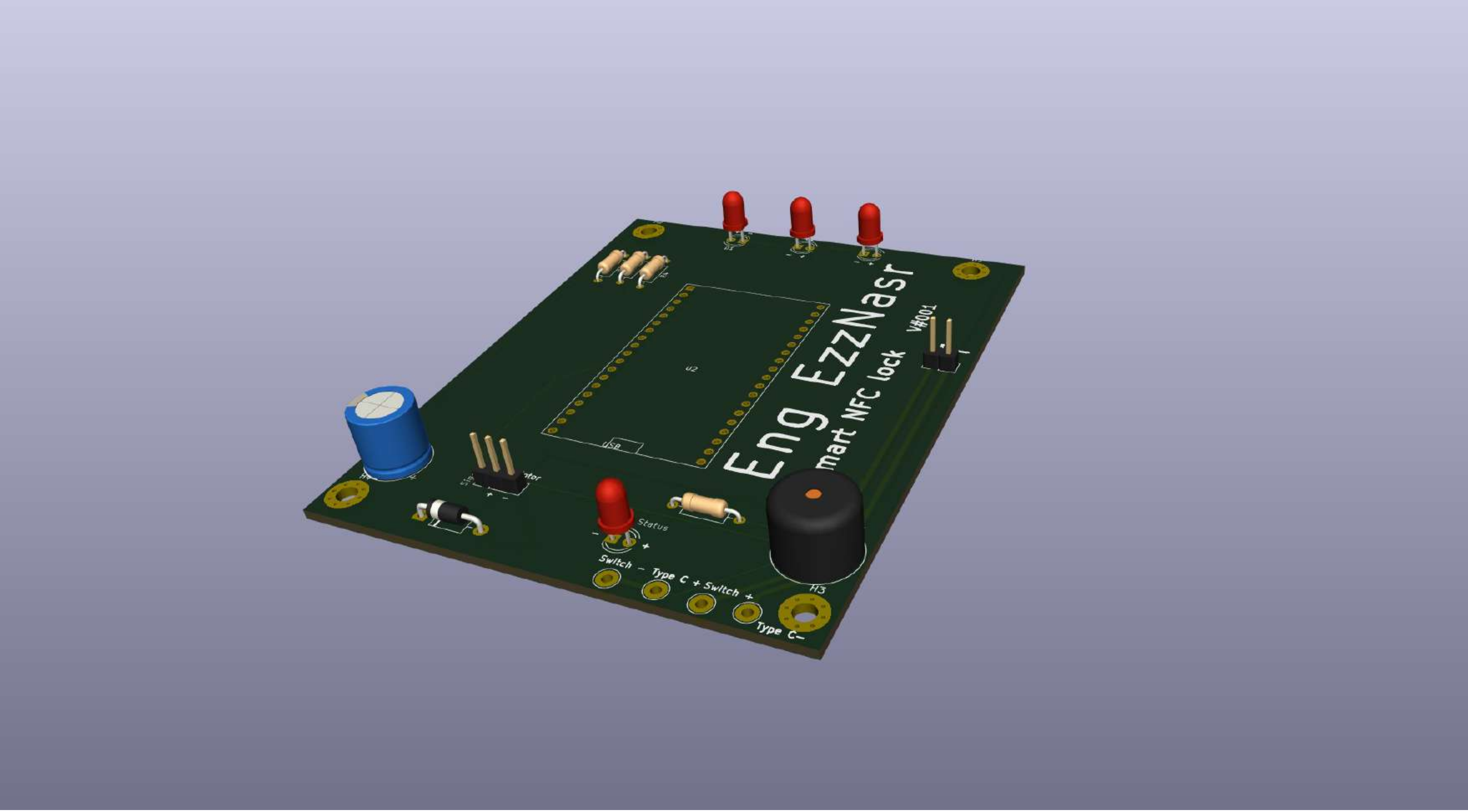
- Gerber Files will be provided
- 3D visualizations will be provided
- schematics will be provided


Eng EzzNasr
Smart NFC lock
V#001
H1
H2
D1
R2
R3
R4
U2
USB
R5
Servo Motor
Signal
Power Status
K
D4
Type C
Switch
Type C-


2/8/2025
-
+

- this PCB implementation features alternative power rails in Case of failure of the USB C port you may use the pins on the top right indicated by **+** and **-**
- we are using a 2 layer PCB measuring under 100mm x 100mm such that the fabrication costs would be relatively attainable
- in addition to being able to route the traces easily
- it was also a concinnous decision not to make the component any closer than this such that it would be relatively easy for a first timer to solder them on their own

in Case you are having trouble locating the files,
it would be my pleasure to answer your questions at
ezznasrone@gmail.com


Eng EzzNasr
mart NFC lock
V#001
U2
USB
Status
Switch − Type C + Switch +
Type C−
H3